

Exercise 3.2 — Backpropagation

Backpropagation is the algorithm used to compute gradients in a computational graph. The core intuition behind a derivative is simply: "how does f change if I slightly modify one of its inputs?" Computing this by hand for large networks would be impractical, so backpropagation automates it.

The algorithm works in two passes. First, a forward pass runs the inputs through the graph and computes the output. Then, a backward pass propagates the gradient of the loss back through each node using the chain rule — from the output all the way to the leaf inputs. Each node only needs to know how to differentiate its own local operation, and the graph handles the rest. In PyTorch, this is what `c.backward()` triggers behind the scenes.

Exercise 3.4: Dataset Exploration

Load the dataset and explore it.

1. How many images are in the training and test splits?
2. What is the class distribution for each label? Are the classes balanced?
3. Display a few example images for each label combination. What patterns do you notice?

1 Training images : 7200

Test images : 3600

Actual train files in rgb-front : 7200

Actual test files in rgb-front : 3600

2. Class distribution (Train)

has_pedestrian:

False (absent) : 5482 (76.1%)

True (present) : 1718 (23.9%)

has_vehicle:

False (absent) : 1742 (24.2%)

True (present) : 5458 (75.8%)

has_traffic_light:

False (absent) : 1924 (26.7%)

True (present) : 5276 (73.3%)

3. From a safety perspective, which metric matters most for each model – precision or recall? Explain.

Recall is the more important metric for all three models, but the reasoning differs:

Model	Most important	Why
Pedestrian	Recall — critically	A missed pedestrian means the autopilot has no signal to trigger emergency braking. The planner relies solely on this output for braking decisions — a false negative directly risks a fatal collision. A false positive just causes unnecessary braking, which is annoying but safe
Traffic light	Recall	Missing a red light means the vehicle drives through it — dangerous for occupants and others. A false positive causes an unnecessary stop, which is a minor inconvenience
Vehicle	Recall	Missing a vehicle ahead risks a rear-end collision. A false positive causes unnecessary caution, which is acceptable

The asymmetry of consequences is the core principle here. In all three cases:

- **False negative** (missed detection) → vehicle fails to respond to a real hazard → **potential harm**
- **False positive** (false alarm) → vehicle responds unnecessarily → **minor inconvenience**

The pedestrian case is the most extreme example of this asymmetry. A recall of 0.010 means the system is functionally blind to pedestrians, it would miss 99 out of 100 real pedestrians in its path. From a safety case perspective, **this model cannot be deployed as-is** and must be fixed before the system operates in any real environment. This is precisely why the pedestrian detector was identified earlier as the most safety-critical component of the three.

Exercise 3.7: Comparing the ODD to the Training Data

In Exercise Sheet 2 you defined an Operational Design Domain (ODD) based on the system description – before seeing any data. Now that you have explored the dataset and trained your models, revisit that ODD.

1. Look at the data you trained on. What conditions does it cover? List the environmental conditions represented in the training set (weather, lighting, scene types, etc.).
2. Compare this to the ODD you defined in Exercise Sheet 2. Which ODD dimensions are not covered by the training data?
3. What are the implications of this gap for your models' safety? For which ODD conditions can you not make any claims about model performance?

4. Note down the gaps you identified (if any). You will revisit them in Sheets 4 and 5 when you formally measure ODD coverage and refine the ODD.

1. What conditions does the training data actually cover?

The CARLA training data covers a narrow, controlled slice of conditions:

Dimension	What the training data contains
Weather	Clear sunny skies; light cloud cover only — no rain, no fog, no snow
Lighting	Daytime only — natural sunlight, no artificial-only illumination, no dusk/dawn
Camera	Forward-facing, rigidly fixed behind windshield; clean, unobstructed lens
Scene type	Urban environments — roads, intersections, pedestrians, vehicles, traffic lights
Speed	Slow-moving or stopped — particularly at intersections; no high-speed scenarios

CARLA renders simplified textures, lighting, and reflections compared to real cameras.

2. Which ODD dimensions are not covered by the training data?

ODD Dimension	Sheet 2 Valid Range	Training Data Coverage	Gap
Weather	Sunny / light overcast	Sunny / light overcast	None
Lighting	Daytime, no glare	Daytime only	Gap: dusk/dawn and direct sun glare are in ODD boundary but absent from training
Camera condition	Clean, unobstructed, 10 Hz	Clean, fixed, stable	Gap: lens contamination scenarios (mud, rain splashes, condensation) not represented
Scene type	Public urban roads, mapped intersections	Urban roads and intersections	Partial gap: motorway, construction zones correctly excluded — but variation within urban (e.g. pedestrian density, roundabouts, road layouts) may be limited

Vehicle speed	0–50 km/h	Slow-moving / stopped	Gap: mid-range and near-limit speeds (e.g. 40–50 km/h) may be underrepresented if data was collected mainly at intersections
---------------	-----------	-----------------------	--

The most significant uncovered dimension is anything involving adverse weather and lighting — rain, fog, night, and glare — but also the implicit assumption that camera condition is always pristine. CARLA's simulator cannot produce lens contamination at all, so no model trained here has ever seen a dirty lens.

3. Safety implications of the gaps

The core implication is that for any ODD condition not represented in the training data, you cannot make any performance claims about models. The three classifiers were trained exclusively on sunny daytime CARLA data. If the vehicle encounters anything outside that, the model outputs are statistically meaningless, the model is operating on inputs drawn from a completely different distribution to the one it learned.

This is not just theoretical. When resnet is trained on night data, the segmentation predictions collapse almost completely. The same would happen to pedestrian, vehicle, and traffic light classifiers. The safety risk is all three perception outputs become unreliable simultaneously, with no sensor redundancy and the human operator as the sole fallback.

There is also the sim-to-real gap as a second layer of concern: even within the training-covered conditions (sunny daytime), the simplified CARLA renderer means models may have learned simulator-specific visual artifacts — edge patterns, colour palettes, reflection behaviour — that don't generalise to real cameras even under identical lighting. This means even the "covered" portion of the ODD only has qualified coverage.

The consequence for the safety case is that you cannot currently sign off on your models for deployment in anything other than sunny daytime conditions in a CARLA-like simulator environment.

4. Identified gaps

- GAP-1 Night-time lighting → no training data; model behaviour unknown
- GAP-2 Dusk / dawn / glare → within ODD boundary; no training representation
- GAP-3 Rain / fog / snow → outside declared ODD; no data; ODD monitor needed
- GAP-4 Lens contamination → simulator cannot produce this; gap is structural
- GAP-5 Mid-high speed (40–50 km/h) → possibly underrepresented; latency budget at risk
- GAP-6 Sim-to-real visual domain → affects all conditions, even sunny daytime

